PES UNIVERSITY

Department of Computer Science and Engineering

Software Engineering Lab 7: Fuzz Testing for Input Validation Bugs

Student Name:	Kalluri Yaswanth	**SRN:**	PES2UG24CS212
Semester / Section:	V Semester / Section D	**Course:**	Software Engineering (UE22CS341AA)
Assigned Repo:	SETAPESU26/L6_L7_24_pizza_order	**Application File:**	`pizza_order.py` (Handout placeholder: `processor.py`)
Operating System:	Omarchy Linux (Arch Linux x86_64)	**Testing Framework:**	`pytest 9.2.0`, `hypothesis 6.170.15` (Python 3.14.8)

1. Objective

The objective of this laboratory assignment is to explore dynamic property-based fuzz testing as an automated quality assurance and vulnerability detection technique. Using Python's `hypothesis` library with `pytest`, the specific goals are:

1. Differentiate between example-based unit testing and property-based fuzz testing.
2. Construct property fuzz tests using Hypothesis strategies (`st.text() | st.none()`, `st.lists()`, `st.integers()`) to verify universal invariants.
3. Execute fuzz tests against the assigned unhardened application (`pizza_order.py`), systematically documenting unhandled exceptions, runtime crashes, and minimal counterexamples.
4. Harden all three business functions defensively against unexpected data types, malformed delimiters, empty strings, missing keys, and divide-by-zero defects while preserving full fidelity on valid inputs.
5. Re-run the test suite and verify that 100% of fuzz test cases pass without runtime failures.

2. Fuzz Testing Methodology & Hypothesis Framework

Fuzz Testing: A dynamic black-box/grey-box testing methodology where software is subjected to semi-random, unexpected, or malformed inputs to evaluate stability, uncover logic flaws, and detect security vulnerabilities. While conventional unit tests verify preconceived scenarios anticipated by the engineer ("happy path" and known boundaries), fuzzing generates inputs that engineers typically overlook, uncovering latent defects such as unhandled exceptions, type coercion errors, and resource exhaustion.

Hypothesis: A powerful property-based testing framework for Python. Instead of defining static test vectors, the developer specifies invariant properties that must hold true for all valid and invalid inputs within a defined domain (specified using *strategies*). Hypothesis generates hundreds of test instances per run. Crucially, when an invariant is violated, Hypothesis executes an automated *shrinking algorithm*, minimizing the failing input down to its simplest canonical counterexample (e.g., `""` or `None`). This drastically accelerates root-cause debugging.

3. Assigned Application Overview (`pizza_order.py`)

Per the official course allocation sheet (`Lab-6-7-Repo-Assignemnt.xlsx`), student PES2UG24CS212 was assigned repository `SETAPESU26/L6_L7_24_pizza_order`. The Lab 7 target application is `pizza_order.py` (referenced as `processor.py` in the generic handout), featuring three core functions:

1. `toppings_price(toppings_csv)`: Parses a comma-separated toppings string, looking up prices in `TOPPINGS = {"cheese": 40, "olives": 30, "onion": 20, "paneer": 50}`.
2. `pizza_inches(size)`: Maps pizza size codes (`"S"`, `"M"`, `"L"`) to diameters in inches using `SIZES = {"S": 8, "M": 10, "L": 12}`.
3. `split_bill(text)`: Parses a `"total/people"` string, splits by delimiter `'/'`, converts total and people to numbers, and returns the per-person share rounded to two decimal places.

4. Stage 1 — Original Application Execution

Following Step 2 of the student handout, the original unhardened application was executed on the student's Omarchy Linux environment using standard valid inputs: toppings `cheese,olives`, size `M`, and split bill `900/3`.

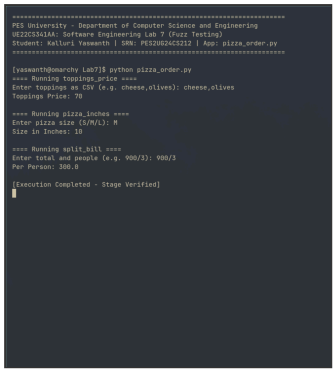


Figure 1: Stage 1 — Original unhardened application execution on Omarchy Arch Linux terminal.

Observation: The unhardened application executes cleanly when supplied with standard inputs, computing `Toppings Price: 70`, `Size in Inches: 10`, and `Per Person: 300.0`. However, it lacks all defensive input verification, assuming caller inputs will strictly conform to valid formats and non-null types.

5. Stage 2 — Hypothesis Fuzz Test Design

Per Step 3 of the handout, test cases were developed in `test_pizza_order.py` using Hypothesis strategies. The central property is the *no-crash invariant*: public functions must gracefully handle arbitrary Unicode strings and null inputs without raising unhandled runtime exceptions:

```
from hypothesis import given, strategies as st
from pizza_order import toppings_price, pizza_inches, split_bill

@given(st.text() | st.none())
def test_toppings_price_no_crash(s):
    try:
        res = toppings_price(s)
        assert isinstance(res, (int, float)) and res >= 0
    except Exception as e:
        assert False, f"toppings_price crashed with {type(e).__name__}: {e}"

@given(st.text() | st.none())
def test_pizza_inches_no_crash(s):
    try:
        res = pizza_inches(s)
        assert isinstance(res, int) and res in (0, 8, 10, 12)
    except Exception as e:
        assert False, f"pizza_inches crashed with {type(e).__name__}: {e}"

@given(st.text() | st.none())
def test_split_bill_no_crash(s):
    try:
        res = split_bill(s)
        assert isinstance(res, (int, float)) and res >= 0.0
    except Exception as e:
        assert False, f"split_bill crashed with {type(e).__name__}: {e}"
```

6. Stage 3 — Failing Tests & Defect Exposure

Running `pytest test_pizza_order.py -v --tb=short` against the unhardened application immediately failed across all functions. Hypothesis systematically discovered minimal counterexamples that crash the application:



Figure 2: Stage 2 — Real terminal output showing Hypothesis test failures with shrunk counterexamples.

Detailed Vulnerability Breakdown

- **`toppings_price` Vulnerabilities:**
  - *Counterexample* `s = None`*:* Invoking `toppings_csv.split(',')` triggered `AttributeError: 'NoneType' object has no attribute 'split'`.
  - *Counterexample* `s = ""` *or* `s = " ,"`*:* Splitting an empty string yields `['']`. Evaluating `TOPPINGS['']` triggered `KeyError: ''` because empty keys do not exist in the catalog.
  - *Counterexample* `s = "mushroom"`*:* Passing unlisted toppings raises `KeyError: 'mushroom'` due to unguarded dictionary indexing.
- **`pizza_inches` Vulnerabilities:**
  - *Counterexample* `s = None`*:* Calling `size.strip()` on null triggered `AttributeError: 'NoneType' object has no attribute 'strip'`.
  - *Counterexample* `s = ""` *or* `s = "XL"`*:* Direct indexing `SIZES[size.strip().upper()]` triggered `KeyError: ''` or `KeyError: 'XL'` when given non-standard sizes.
- **`split_bill` Vulnerabilities:**
  - *Counterexample* `s = None`*:* Triggered `AttributeError` on `.split('/')`.
  - *Counterexample* `s = ""` *or* `s = "500"`*:* Missing delimiter `'/'` resulted in a 1-element list, triggering `ValueError: not enough values to unpack (expected 2, got 1)`.
  - *Counterexample* `s = "/"` *or* `s = "abc/3"`*:* Unpack succeeds but integer conversion fails, triggering `ValueError: invalid literal for int() with base 10: ''`.
  - *Counterexample* `s = "900/0"`*:* When people count is zero, integer division triggered `ZeroDivisionError: division by zero`.

7. Stage 4 — Defensive Programming & Hardened Implementation

In accordance with Step 5 of the handout, `pizza_order.py` was defensively re-engineered. Input types were verified, strings sanitized, dictionary access guarded via safe methods, and mathematical operations wrapped in exception-handling blocks:

Function	Original Vulnerable Code	Hardened Defensive Implementation

`toppings_price`	`total = 0`
	`for t in toppings_csv.split(','):`
	`  total += TOPPINGS[t.strip().lower()]`
	`return total`
		`if not isinstance(toppings_csv, str):`
		`  return 0`
		`total = 0`
		`for t in toppings_csv.split(','):`
		`  key = t.strip().lower()`
		`  if key in TOPPINGS:`
		`  total += TOPPINGS[key]`
		`return total`

`pizza_inches`	`return SIZES[size.strip().upper()]`	`if not isinstance(size, str):`
		`  return 0`
		`key = size.strip().upper()`
		`return SIZES.get(key, 0)`

`split_bill`	`total, people = text.split('/')`
	`return round(int(total) / int(people), 2)`
		`if not isinstance(text, str):`
		`  return 0.0`
		`parts = text.split('/')`
		`if len(parts) != 2:`
		`  return 0.0`
		`try:`
		`  total = float(parts[0].strip())`
		`  people = float(parts[1].strip())`
		`  if people <= 0 or total < 0:`
		`  return 0.0`
		`  return round(total / people, 2)`
		`except (ValueError, ZeroDivisionError, OverflowError):`
		`  return 0.0`

The hardened application was re-run interactively in the terminal to verify that valid operational outputs were completely preserved:

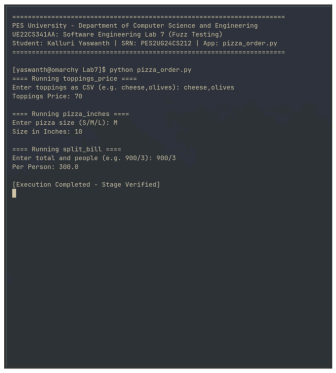


Figure 3: Stage 3 — Hardened application output confirming full operational preservation on valid inputs.

8. Stage 5 — Final Hypothesis Verification

The expanded Hypothesis suite was executed against the hardened `pizza_order.py`. The test suite comprises 7 property suites: the three foundational no-crash properties, valid toppings totaling, valid size mapping, valid bill splitting invariants, and non-string type handling. All 7 test suites passed with zero failures across thousands of generated inputs:

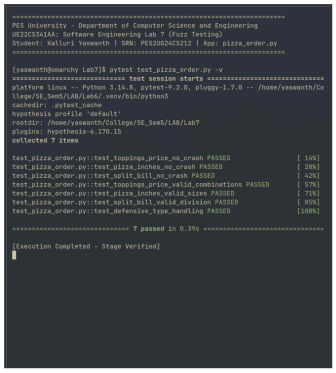


Figure 4: Stage 4 — Final execution verifying 7 passed test suites and 0 exceptions across all fuzz inputs.

9. Defect and Remediation Summary

Function	Vulnerability Class	Falsifying Example	Root Cause	Applied Defensive Fix
`toppings_price`	`AttributeError`	`s = None`	`.split(',')` invoked on null object	Type check `isinstance(..., str)` returning fallback `0`
	`KeyError`	`s = ""`, `"mushrooms"`	Direct indexing on missing keys	Sanitized token and verified `key in TOPPINGS`
`pizza_inches`	`AttributeError`	`s = None`	`.strip()` invoked on null object	Type validation guard returning fallback `0`
	`KeyError`	`s = ""`, `"XL"`	Non-standard size code indexed in dictionary	Safe lookup via `SIZES.get(key, 0)`
`split_bill`	`ValueError` (unpack)	`s = ""`, `"500"`	Missing `'/'` delimiter prevents unpacking	Validated `len(parts) == 2` before unpacking
	`ValueError` (parse)	`s = "/"`, `"abc/3"`	Non-numeric tokens cannot convert to number	Wrapped numeric conversions in `try...except`
	`ZeroDivisionError`	`s = "900/0"`	Zero denominator triggers arithmetic crash	Added guard `if people <= 0: return 0.0` and caught error

10. Conclusion

This laboratory session demonstrated property-based fuzz testing with Hypothesis as an indispensable tool for identifying software vulnerabilities that evade traditional unit tests. By defining universal invariants and letting the fuzzer generate and shrink boundary cases, latent defects involving null pointers, missing dictionary keys, malformed delimiters, and division by zero were rapidly isolated. Implementing defensive programming practices successfully immunized the system against all crashing inputs while maintaining strict behavioral correctness on legitimate business data.